

Constant Pointers and Pointers to Constant

A Subtle Difference in C Programming

Aimed at those new to C programming, this article clears up the confusion between the terms used in it, with illustrative examples.



Pointers have always been a complex topic to understand for those new to C programming. There will be more confusion for newbies when these terms are used along with some qualifiers like `const` in C programming. In this article, I will focus on the difference between the 'pointers to constant' and 'constant pointers' in order to make the concepts very clear.



Note: The code snippets provided here have been tested with the GCC compiler [gcc version 4.8.2] running under the Linux environment.

Pointer to constant

As the name itself indicates, the value of the variable to which the pointer is pointing, is constant. In other words, a pointer through which one cannot change the value of the variable to which it points is known as a pointer to constant.



Note: These pointers can change the address they point to but cannot change the value at the address they are pointing to.

Illustration 1

Let us consider the code snippet given below to understand

Table 1: Syntax to declare the pointer to constant

Syntax	Example
<code>const <type of pointer>*<pointer name></code>	<code>const int*ptr</code>
OR	
<code><type of pointer>const*<pointer name></code>	<code>int const*ptr</code>

how pointer to constant works:

```

1 #include <stdio.h>
2
3 int main()
4 {
5     //Definition of the variable
6     int a = 10;
7
8     //Definition of pointer to constant
9     const int* ptr = &a; //Now, ptr is pointing to
    the value of the variable 'a'
10
11     *ptr = 30; //Error: Since the value is constant
12
13     return 0;
14 }
```